

Garbage Collector

What and Why?

- Memory management process.
- Makes Java memory efficient

Few ways when the object is marked as dead..

Dereference

```
Student student = new Student();  
student = null;
```

```
Student studentOne = new Student("Smriti");  
Student studentTwo = new Student("Rob");  
studentOne = studentTwo;
```

```
new Student();
```

Scope

```
void someMethod() {  
  
    String str = new String("abc");  
  
}
```

3 Phases of GC

INITIAL :



1. MARK:



2. SWEEP :

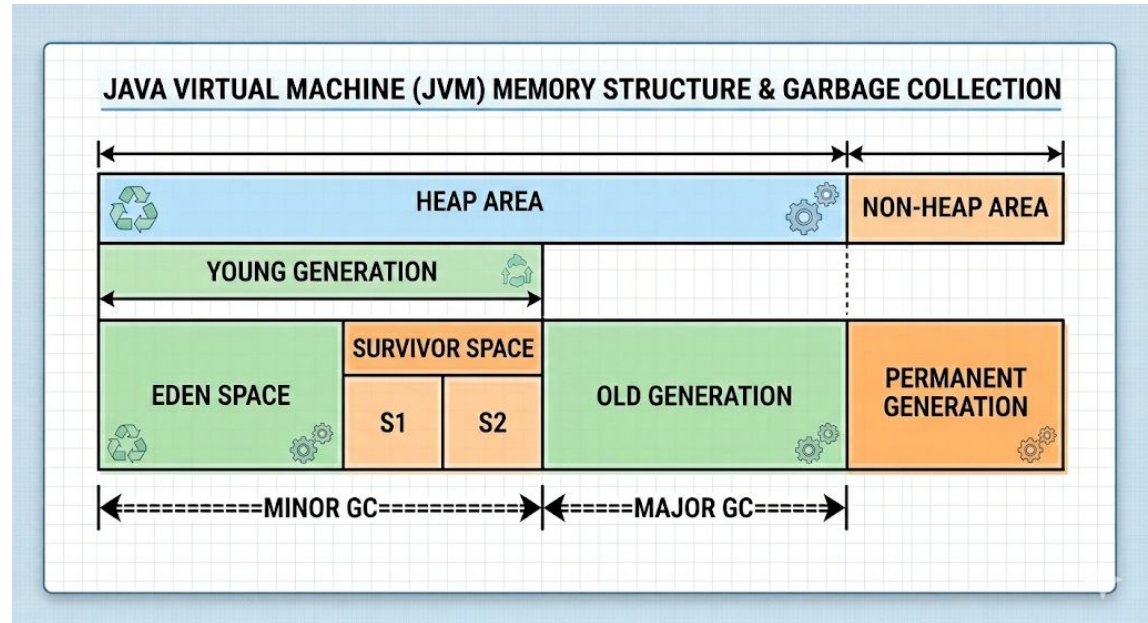


3. COMPACT :



The Heap memory

- Minor Garbage Collection Event
- Major Garbage Collection Event
- Permanent Generation



GC Types

Serial GC

- One thread
- Stop the world

Parallel GC / Parallel Old

- Multiple threads
- Stop the world

CMS (Concurrent Mark Sweep) GC

- Concurrently alongside application, Multiple threads
- Stop the world
- ***No compaction*** is performed in CMS
- ***Deprecated in Java 9***

Epsilon

- Its ONLY job is memory allocation
- JVM shutdowns as Java heap is exhausted

G1 (Garbage First) GC

- Partitions heap and allocates thread to scan them
- GC performed based on used preference
- Still causes "stop the world"
- Pause duration for memory reclamation is controllable

Shenandoah

- Concurrently with the application threads allowing it to work with live objects
- Release RAM back to the OS almost immediately
- Resolves "stop the world"

ZGC

- Runs in the background too and works without pausing the application
- Divides the heap into small chunks
- Finishes marking dead objects phase within 1ms.
- Any region that contains no live objects is reclaimed immediately

Manual Triggers

```
System.gc();
```

```
Runtime.gc();
```